

DIGITAL LOGIC DESIGN LAB

Verilog HDL Programs & Testbenches

Experiments: Half Adder, Full Adder, 3:8 Decoder, D Latch, D Flip-Flop, 2:1 MUX, and 4:1 MUX.

Vivado note: Add each design module as a Design Source and its corresponding *_tb module as a Simulation Source. Run Behavioral Simulation to obtain the waveform.

Experiment 1 — Half Adder

Aim: To design and simulate a Half Adder using Verilog HDL.

Logic / Schematic:

$A, B \rightarrow [\text{HALF ADDER}] \rightarrow \text{SUM}, \text{CARRY}$

Verilog Design Code:

```
module half_adder(  
    input a,  
    input b,  
    output sum,  
    output carry  
);  
  
assign sum = a ^ b;  
assign carry = a & b;  
  
endmodule
```

Testbench Code:

```
module half_adder_tb;  
  
    reg a, b;  
    wire sum, carry;  
  
    half_adder uut(a, b, sum, carry);  
  
    initial  
    begin  
        a = 0; b = 0; #10;  
        a = 0; b = 1; #10;  
        a = 1; b = 0; #10;  
        a = 1; b = 1; #10;  
  
        $finish;  
    end  
  
endmodule
```

Truth Table / Expected Behavior:

A	B	SUM	CARRY
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Result: The Half Adder was successfully designed and simulated using Verilog HDL.

Experiment 2 — Full Adder

Aim: To design and simulate a Full Adder using Verilog HDL.

Logic / Schematic:

A, B, Cin → [FULL ADDER] → SUM, CARRY

Verilog Design Code:

```
module half_adder(
    input a,
    input b,
    output sum,
    output carry
);

assign sum  = a ^ b;
assign carry = a & b;

endmodule

module full_adder(
    input a,
    input b,
    input cin,
    output sum,
    output carry
);

wire sum1, carry1, carry2;

half_adder HA1(a, b, sum1, carry1);
half_adder HA2(sum1, cin, sum, carry2);
assign carry = carry1 | carry2;

endmodule
```

Testbench Code:

```
module full_adder_tb;

reg a, b, cin;
wire sum, carry;

full_adder uut(a, b, cin, sum, carry);

initial
begin
    a=0; b=0; cin=0; #10;
    a=0; b=0; cin=1; #10;
    a=0; b=1; cin=0; #10;
    a=0; b=1; cin=1; #10;
    a=1; b=0; cin=0; #10;
    a=1; b=0; cin=1; #10;
    a=1; b=1; cin=0; #10;
    a=1; b=1; cin=1; #10;

    $finish;
end

endmodule
```

A	B	Cin	SUM	CARRY
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Truth Table / Expected Behavior:

Result: The Full Adder was successfully designed and simulated using Verilog HDL.

Experiment 3 — 3:8 Decoder

Aim: To design and simulate a 3:8 Decoder using Verilog HDL.

Logic / Schematic:

$A_2, A_1, A_0 \rightarrow [3:8 \text{ DECODER}] \rightarrow Y_0 \dots Y_7$

Verilog Design Code:

```
module decoder3to8(
    input  [2:0] a,
    output reg [7:0] y
);

always @(*)
begin
    case(a)
        3'b000: y = 8'b00000001;
        3'b001: y = 8'b00000010;
        3'b010: y = 8'b00000100;
        3'b011: y = 8'b00001000;
        3'b100: y = 8'b00010000;
        3'b101: y = 8'b00100000;
        3'b110: y = 8'b01000000;
        3'b111: y = 8'b10000000;
        default: y = 8'b00000000;
    endcase
end

endmodule
```

Testbench Code:

```
module decoder3to8_tb;

reg [2:0] a;
wire [7:0] y;

decoder3to8 uut(a, y);

initial
begin
    a = 3'b000; #10;
    a = 3'b001; #10;
    a = 3'b010; #10;
    a = 3'b011; #10;
    a = 3'b100; #10;
    a = 3'b101; #10;
    a = 3'b110; #10;
    a = 3'b111; #10;

    $finish;
end

endmodule
```

Truth Table / Expected Behavior:

A2	A1	A0	Y7 Y6 Y5 Y4 Y3 Y2 Y1 Y0
0	0	0	00000001
0	0	1	00000010
0	1	0	00000100
0	1	1	00001000
1	0	0	00010000
1	0	1	00100000
1	1	0	01000000
1	1	1	10000000

Result: The 3:8 Decoder was successfully designed and simulated using Verilog HDL.

Experiment 4 — D Latch

Aim: To design and simulate a D Latch using Verilog HDL.

Logic / Schematic:

$D, EN \rightarrow [D \text{ LATCH}] \rightarrow Q$

Verilog Design Code:

```
module d_latch(  
    input d,  
    input en,  
    output reg q  
);  
  
always @(*)  
begin  
    if(en)  
        q = d;  
end  
  
endmodule
```

Testbench Code:

```
module d_latch_tb;  
  
    reg d, en;  
    wire q;  
  
    d_latch uut(d, en, q);  
  
    initial  
    begin  
        en = 0; d = 0; #10;  
        en = 0; d = 1; #10;  
        en = 1; d = 0; #10;  
        en = 1; d = 1; #10;  
        en = 0; d = 0; #10;  
        en = 0; d = 1; #10;  
  
        $finish;  
    end  
  
endmodule
```

Truth Table / Expected Behavior:

EN	D	Q (next)
0	0	Q(previous)
0	1	Q(previous)
1	0	0
1	1	1

Result: The D Latch was successfully designed and simulated using Verilog HDL.

Experiment 5 — D Flip-Flop

Aim: To design and simulate a positive-edge triggered D Flip-Flop using Verilog HDL.

Logic / Schematic:

D, CLK, RESET → [D FLIP-FLOP] → Q

Verilog Design Code:

```
module dff(  
    input d,  
    input reset,  
    input clk,  
    output reg q  
);  
  
always @(posedge clk)  
begin  
    if(reset)  
        q <= 1'b0;  
    else  
        q <= d;  
    end  
end  
  
endmodule
```

Testbench Code:

```
module dff_tb;  
  
    reg d, reset, clk;  
    wire q;  
  
    dff uut(d, reset, clk, q);  
  
    initial  
    begin  
        clk = 0;  
        reset = 1;  
        d = 0;  
  
        #10;  
        reset = 0;  
        d = 0;  
  
        #10;  
        d = 1;  
  
        #10;  
        d = 0;  
  
        #10;  
        d = 1;  
  
        #10;  
        d = 0;  
  
        #10;  
        $finish;  
    end  
  
    always #5 clk = ~clk;  
  
endmodule
```

Truth Table / Expected Behavior:

CLK	D	RESET	Q (next)
↑	0	0	0
↑	1	0	1
↑	0	1	0
No ↑	X	0	Q(previous)

Result: The D Flip-Flop was successfully designed and simulated using Verilog HDL.

Experiment 6 — 2:1 Multiplexer

Aim: To design and simulate a 2:1 Multiplexer using Verilog HDL.

Logic / Schematic:

$A, B, S \rightarrow [2:1 \text{ MUX}] \rightarrow Y$

Verilog Design Code:

```
module mux2to1(
    input a,
    input b,
    input s,
    output y
);

assign y = s ? b : a;

endmodule
```

Testbench Code:

```
module mux2to1_tb;

    reg a, b, s;
    wire y;

    mux2to1 uut(a, b, s, y);

    initial
    begin
        s=0; a=0; b=0; #10;
        s=0; a=0; b=1; #10;
        s=0; a=1; b=0; #10;
        s=0; a=1; b=1; #10;
        s=1; a=0; b=0; #10;
        s=1; a=0; b=1; #10;
        s=1; a=1; b=0; #10;
        s=1; a=1; b=1; #10;

        $finish;
    end

endmodule
```

Truth Table / Expected Behavior:

S	A	B	Y
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

Result: The 2:1 Multiplexer was successfully designed and simulated using Verilog HDL.

Experiment 7 — 4:1 Multiplexer using 2:1 MUXes

Aim: To design and simulate a 4:1 Multiplexer using three 2:1 Multiplexers.

Logic / Schematic:

$A, B \rightarrow M1; C, D \rightarrow M2; M1, M2 \rightarrow M3 \rightarrow Y$

Verilog Design Code:

```
module mux2to1(
    input a,
    input b,
    input s,
    output y
);

assign y = s ? b : a;

endmodule

module mux4to1(
    input a,
    input b,
    input c,
    input d,
    input s1,
    input s2,
    output y
);

wire y1, y2;

mux2to1 m1(a, b, s1, y1);
mux2to1 m2(c, d, s1, y2);
mux2to1 m3(y1, y2, s2, y);

endmodule
```

Testbench Code:

```
module mux4to1_tb;

reg a, b, c, d;
reg s1, s2;
wire y;

mux4to1 uut(a, b, c, d, s1, s2, y);

initial
begin
    a=0; b=1; c=0; d=1;

    s2=0; s1=0; #20;
    s2=0; s1=1; #20;
    s2=1; s1=0; #20;
    s2=1; s1=1; #20;

    $finish;
end

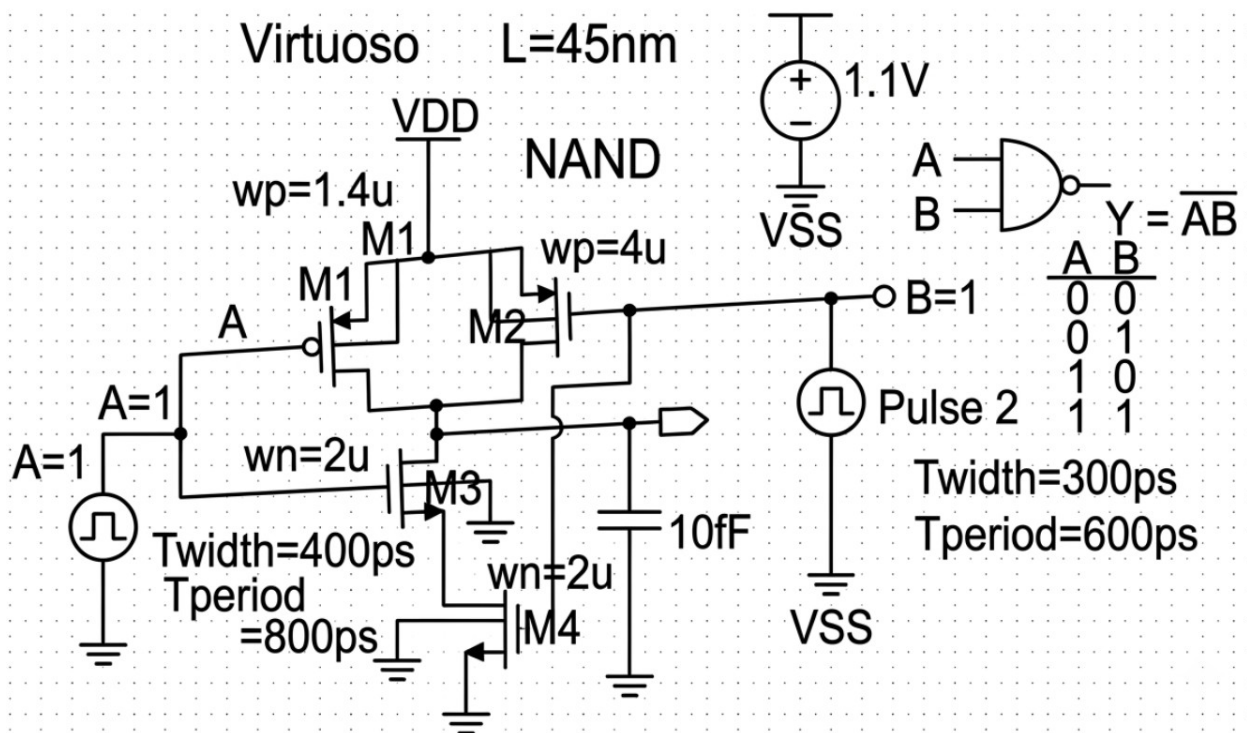
endmodule
```

Truth Table / Expected Behavior:

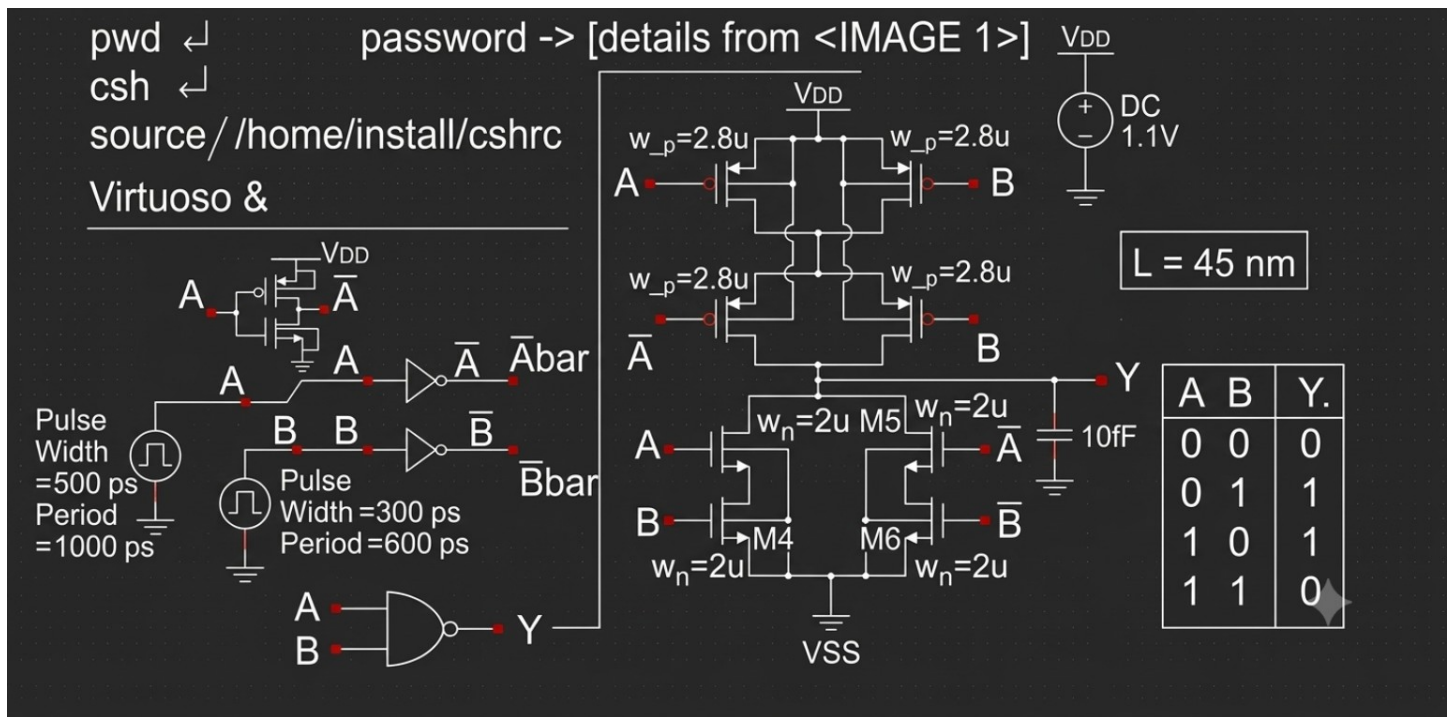
S2	S1	Y
0	0	A
0	1	B
1	0	C
1	1	D

Result: The 4:1 Multiplexer was successfully designed and simulated using three 2:1 Multiplexers.

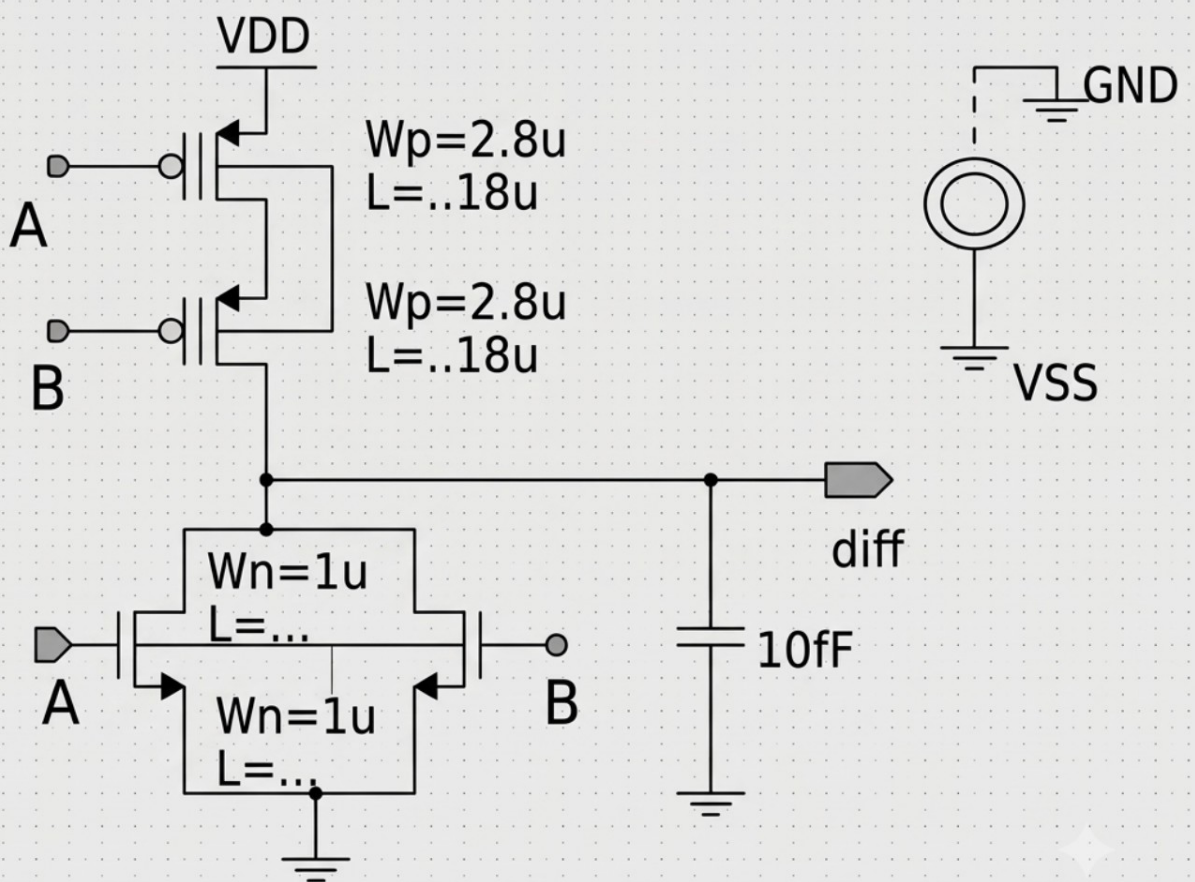
Realisation of Transient Analysis of CMOS NAND Gate



Realisation of CMOS 2-Input XOR Gate (Transient Analysis)



Realisation of CMOS 2-Input NOR Gate (Transient Analysis)



Realisation of Transient Analysis of CMOS Inverter

Schematic Diagram of CMOS Inverter (For Transient Analysis):

